

Asymptotic

An incremental audit and a
uniform Lerch extension

A reflected-Erfc frontier defect,
a reproduced unsupported parameter regime,
and an additive implementation with a signed bound

Repository VladimirReshetnikov / Asymptotic

Reviewed revision

ef1a1e1c4845a9c35e140963a0333d0c9ec33b05

Review date 10 September 2026

Targets Wolfram Language 15 and Mathics3

Evidence boundary. Targeted execution succeeded in Wolfram Language 15.0.0. The independent Python suite passed 12 test methods, including 96 numerical signed-bound subcases. No Mathics execution or complete upstream acceptance run was performed. The supplied patch was checked in memory on one native reproduction; the distributed integration-test file remains unexecuted.

Contents

1	Executive assessment	2
2	Revision, method, and evidence discipline	3
2.1	The source snapshot	3
2.2	What was actually executed	3
2.3	An exploratory oracle that was rejected	4
2.4	Non-duplication policy	4
3	Source inspection and negative results	4
4	F01: the reflected Erfc frontier has the wrong sign	6
4.1	Minimal public reproduction	6
4.2	Independent derivation of the correction	6
4.3	Root cause and affected contract	7
4.4	Minimal patch	7
4.5	Independent numerical evidence	8
4.6	Integration acceptance for F01	8
5	G01: a coupled Lerch limit is conservatively unsupported	8
5.1	The reproduced request	8
5.2	Why the existing fixed-parameter series cannot just be reused	9
6	E01: a uniform expansion with a signed next-term bound	9
6.1	Definitions and result	9
6.2	A scalar kernel inequality	10
6.3	Proof of the Lerch theorem	11
6.4	Uniformity and boundary checks	12
6.5	A fully explicit example	12
7	Implementation and performance implications	13
7.1	The additive interface	13
7.2	Coefficient construction	13
7.3	Work as the large parameter grows	13
7.4	Numerical stability of the leading term	14
7.5	A bounded integration path	14
8	Wolfram 15 and Mathics3 assessment	14
9	Priorities and concrete deliverables	15
A	Artifact inventory and reproduction	15
B	The independent test matrix	16
	Revision efa1aee	1

1 Executive assessment

The principal new defect established in this review is narrow but definite: the negative-source `Erfc` inverse adapter reflects its finite approximation and its displayed terms, but inherits an unreflected, signed `FrontierTerm`. A consumer that adds that field as the next correction moves in the wrong direction. The stored magnitude remainder is not the part that needs negation. The minimal repair adds a single reflected field at the adapter's object-assembly boundary, preserving an unknown frontier as `Missing` rather than multiplying it by a sign. Both the original behavior and the one-case in-memory repair were observed in Wolfram 15.0.0. The responsible canonical source is `SpecialFunctionAdapters.wl`, particularly lines 47–68.[3]

A second result is a concrete, conservatively rejected input rather than a wrong answer. The current package does not produce an expansion for

$$\Phi(e^{-1/x}, 2, x), \quad x \longrightarrow +\infty,$$

under the tested explicit package-backend request. It returns `Failure["UnsupportedNativeCo efficient", ...]`. This is a parameter-transition problem: the first Lerch parameter approaches one while the third grows. The dedicated provider admits a fixed first parameter, so simply relaxing its admission guard would not be a sound fix. Its fixed-parameter expansion changes asymptotic character in this regime. [4]

The report supplies an additive implementation proposal for that missing regime. For fixed $\lambda > 0$, $s \geq 0$, and integer $K \geq 0$, it constructs an expansion of $\Phi(e^{-\lambda/a}, s, a)$ with a globally valid, signed next-term remainder bound. Its leading term is

$$a^{1-s} e^\lambda \lambda^{s-1} \Gamma(1-s, \lambda),$$

not the leading term obtained by substituting a moving parameter into the fixed-parameter series. The article proves the bound and gives exact-rational coefficient generation, a Python evaluator, a Wolfram Language prototype, and proposed integration tests. These are companion artifacts, not changes already merged into the repository.

ID	Classification	Evidence and disposition
F01	Signed-frontier correctness defect	Reproduced natively; minimal in-memory patch checked. Fix before relying on reflected frontiers.
G01	Unsupported parameter regime	One explicit public request reproduced as a conservative failure. Not a false expansion.
E01	Proposed solution to G01	Full analytic derivation, additive code, independent exact and numerical tests, and a native coefficient check.

G01 and E01 are the gap and its proposed solution, not two independent bugs. No claim is made that this review has established the absence of other defects. In particular, one successful native load is not a Wolfram-wide acceptance result, and source-level portability is not a Mathics execution result.

The current review directory already contains a substantial accumulated body of analysis: its index organizes 45 retained reports into six waves. This report deliberately does not republish that backlog. The conclusions above were selected after comparing the maintained indexes and relevant nearby ledgers, and after inspecting substantial portions of twenty canonical modules. The remainder of this article explains what that evidence does and does not establish.[1]

2 Revision, method, and evidence discipline

2.1 The source snapshot

The review is tied to efa1aeec4845a9c35e140963a0333d0c9ec33b05, not to a moving main URL. The native probes imported the generated root `AsymptoticAnalysis.wl` from the commit-pinned raw URL. Canonical-source inspection used the corresponding `src/Kernel` files through the GitHub connector. No repository write, pull request, or external commit was made.

A local clone and local network downloads were unavailable in the working container. Consequently, there was no local checkout on which to run a complete build or source-wide test command. The connector nevertheless provided the source passages used in the analysis, and the native service successfully fetched and evaluated the pinned standalone on several calls. Other native calls failed at the service or network layer. Those failures are not package failures and are excluded from the mathematical conclusions.

The native environment identified itself as

```
15.0.0forLinuxx86(64-bit)(May6,2026).
```

The local independent tests used Python 3.13.5, SymPy 1.14.0, and mpmath 1.3.0. Exact environment details are retained in `evidence/environment.json`. The repository's own Mathics compatibility document names Mathics3 10.0.1, scanner 10.0.1, SymPy 1.14.0, and Python 3.11 as its development configuration; that is a statement about the pinned project's documentation, not a new run of that environment.[\[11\]](#)

2.2 What was actually executed

A native load control produced $y - y^2 + 2y^3$ for the inverse of $x + x^2$ through exclusive target cutoff four. The original reflected-Erfc probe returned the incorrectly oriented frontier shown in Section 4. Replacing the single object-assembly anchor in the fetched standalone text and rerunning that one probe changed only the frontier's sign among the three fields observed. The unsupported Lerch transition request was also executed.

For the proposed Lerch model, a fully qualified native coefficient routine returned the expected exact terms and bound, and a thirty-digit comparison at $a = 10$ agreed with the independent implementation. The complete distributed `UniformLerch.wl` file was not loaded as a file in that service, and the complete `ReviewProbes.wlt` file was not run. An initial declaration attempt suffered a parsing-context error and was discarded; it supplied no prototype acceptance evidence.

The local suite executed twelve `unittest` methods. These include exact rational identities, symbolic coefficient equations, patch-generation guards, input validation, and 96 numerical checks of the signed Lerch bound. They are *independent companion tests*, not twelve upstream tests and not 96 native-package acceptance cases.

Native observations in the archive are explicitly labeled manual transcriptions of returned values. They are not downloaded machine-verifiable service receipts. This distinction matters: a reproducible input and an exact reported output are useful evidence, but should not be silently converted into a claim about the repository's own acceptance ledger.

2.3 An exploratory oracle that was rejected

A twenty-four-expression forward screen considered elementary functions of $x + x^n$, $n = 2, 3, 4$. Eighteen integral-power cases produced no low-order discrepancy in that screen. Six fractional-power cases yielded invalid oracle flags because the comparator substituted $x = t^2$ without establishing $t > 0$ and then attempted polynomial coefficient extraction on expressions whose form did not support it. Those flags were discarded.

This is an important limitation, not an upstream defect. The report does not count those six flags as wrong package results and does not count the full screen as twenty-four passing tests. Diagnostic messages from that exploratory call were not independently isolated, so no additional UX finding is made from them. The test transcript describes the rejected oracle rather than hiding it.

2.4 Non-duplication policy

The exclusion comparison used the root review index, all six wave indexes, the central development register and selected nearby original ledgers. It was not a word-for-word reading of every historical article and attachment. No absolute absence-of-anticipation claim is therefore appropriate. The claim-specific distinctions are as follows.[1, 2]

This item	Nearest prior material	Different mechanism or regime
F01	Wave 5, report 43, F01	That item concerns coefficient reconstruction through <code>InverseExpansionCoefficient</code> . Here the <code>Erfc</code> wrapper already reflects its approximation correctly, but copies a separate signed field without reflection. The coefficient-query path is not involved.
G01/E01	Wave 6, report 49, E1	The nearby signed Lerch construction concerns a fixed geometric parameter. Here $z = e^{-\lambda/a}$ varies with a , the leading power changes, and a Gamma-moment resummation is required.
G01/E01	Wave 6, reports 50 and 55	Fixed-parameter moment evaluation and provider dispatch/closure are different from this coupled parameter limit. The proposal is not to repeat a moment-table optimization or to remove a bare-atom restriction.

The present derivation uses classical identities. “New” here refers to the concrete review contribution relative to the examined repository material, not to a claim of mathematical priority over the literature. [13, 14]

3 Source inspection and negative results

The table below is an inspection map, not a list of newly found defects. “Selected passages” means exactly that; in particular, it does not imply line-by-line inspection of every branch in the named module. A lack of a new finding is not a proof of correctness.

Canonical source area	Inspected mechanisms	Review outcome
AsymptoticAnalysis.wl	Public entry points; local coordinates; sparse jets; precision arithmetic; inverse regions and coefficients; construction; residuals; object assembly	No additional independently established defect published.
SeriesOperations.wl	Selected representation, composition, truncation, differentiation and refinement passages	No new claim about the uninspected arithmetic branches.
SpecialFunctionAdapters.wl	Erfc, Gamma and threshold construction; checks and metadata	F01 isolated to reflected frontier transport.
DirichletSpecialFunctions.wl	Fixed-parameter Lerch moments, admission, Zeta/Lerch object construction and bounds	G01/E01 concerns a distinct coupled limit, not unsoundness of fixed-parameter bounds.
BarnesForward.wl; ParameterizedSpecialFunctions.wl; SpecialFunctionIdentities.wl	Barnes asymptotic coefficients; parameterized Frobenius factors; terminating and half-integer identities	Representative formulas checked algebraically; no new counterexample established.
GammaInverseOperations.wl; GammaInverseChecks.wl	Powered observables and selected independent residual construction	No new defect established; not exhaustive numerical or parameter-domain acceptance.
CorePerturbation.wl; LogarithmicScales.wl	Selected model admission, exact-core identities, marker coefficients and logarithmic-unit construction	No additional publication-quality counterexample.
SourceCoordinates.wl; CoordinateInverse.wl	Source-chart reconstruction and logarithmic target transformations	Inspected chart semantics; no new general transformation claim.
NativeSpecialFunctions.wl	Selected native tree import, error transport and sector admission	The G01 native coefficient failure was traced to an unexpanded moving-parameter coefficient.
MathicsCalls.wl; MathicsAlgebra.wl; MathicsAssumptions.wl; MathicsSimplification.wl; MathicsCalculus.wl; MathicsSpecialFunctions.wl	Selected held extraction, algebraic adapters, realness and sign reasoning Simplification/limit adapters, finite derivative tables and Stirling fallback	Source review only; no Mathics execution claim. No additional new defect established in the passages inspected.

Several candidate concerns did not survive inspection. For example, the ordinary inverse constructor explicitly rejects a relative cutoff that does not exceed its leading observable exponent; an empty multi-index region at such a cutoff is not a valid public counterexample. The precise guard occurs in the construction path before region enumeration. Likewise, ordinary truncation is routed through a derived series representation rather than silently preserving an arbitrary inverse-model interpretation. These are observations about the current source, not recommendations to repair already guarded behavior.[5, 6]

The checked special-function formulas also supplied useful controls. The Barnes logarithm uses the expected Bernoulli coefficient pattern, and the inspected terminating hypergeometric and half-integer Bessel identities did not yield a separate error after accounting for their admission conditions. This review therefore does not turn “complicated code” into a defect claim. [7, 8, 9]

4 F01: the reflected Erfc frontier has the wrong sign

4.1 Minimal public reproduction

After loading the pinned standalone in Wolfram 15.0.0, evaluate:

```
s = AsymptoticAnalysis'AsymptoticSpecialInverse[
  "Erfc", {x, -Infinity}, {y, 1}];
{Normal[s], s["FrontierTerm"], s["RemainderPower"]}
```

Put

$$v = -\log(\sqrt{\pi}(2-y)), \quad L = \log v.$$

The three returned values, with only this editorial abbreviation, are

$$g_-(y) = -\sqrt{v} + \frac{L}{4\sqrt{v}}, \quad (1)$$

$$F_{\text{stored}}(y) = \frac{B(L)}{v^{3/2}}, \quad B(L) = -\frac{1}{4} + \frac{L}{8} - \frac{L^2}{32}, \quad (2)$$

$$\rho = \frac{3}{2}. \quad (3)$$

The finite approximation is correctly reflected. The signed frontier is not. Notice the identity

$$B(L) = -\frac{(L-2)^2 + 4}{32} < 0 \quad (L \in \mathbb{R}).$$

Thus this is not merely an ambiguity at a coefficient zero.

4.2 Independent derivation of the correction

For the positive inverse branch, let $z > 0$ and define $v = -\log(\sqrt{\pi} \operatorname{erfc} z)$. The standard positive-real Erfc asymptotic expansion implies

$$v = z^2 + \log z + \frac{1}{2z^2} + O(z^{-4}). \quad (4)$$

The source of this relation is the expansion of the original special function, not the package's stored frontier.[16]

Substitute

$$z = \sqrt{v} + \frac{A(L)}{\sqrt{v}} + \frac{B(L)}{v^{3/2}} + O\left(\frac{(1+L)^3}{v^{5/2}}\right).$$

Equating the constant and v^{-1} terms in (4) gives

$$2A + \frac{L}{2} = 0, \quad A^2 + 2B + A + \frac{1}{2} = 0.$$

Consequently

$$A = -\frac{L}{4}, \quad B = -\frac{1}{4} + \frac{L}{8} - \frac{L^2}{32}.$$

These two coefficient equations are checked independently in `test_review.py` using exact symbolic arithmetic.

The reflection identity $\operatorname{erfc}(-z) = 2 - \operatorname{erfc}(z)$ now determines the negative inverse branch without a second inversion calculation.^[15] It has expansion

$$\operatorname{erfc}^{-1}(y) = -\sqrt{v} + \frac{L}{4\sqrt{v}} - \frac{B(L)}{v^{3/2}} + O\left(\frac{(1+L)^3}{v^{5/2}}\right), \quad y \rightarrow 2^-.$$
 (5)

The correct signed next block is therefore $-B(L)v^{-3/2}$, the negative of the stored field in (2).

This derivation also explains the practical impact. If a consumer adds the stored frontier to (1), the new error is asymptotic to $-2B(L)v^{-3/2}$ instead of $-B(L)v^{-3/2}$. The leading error is approximately doubled. Adding the corrected frontier eliminates that block.

4.3 Root cause and affected contract

In the canonical adapter, `sign` is 1 at positive source infinity and -1 at negative source infinity. The code multiplies the finite expression and the `Terms` coefficients by that sign. It also reflects the stored series representation's prefactor and offset. It then joins the original `innerData` association with the adapter-specific replacement fields. `FrontierTerm` is not among those replacements, so the inner positive source's signed frontier survives.^[3]

The relevant boundary is not the Lagrange coefficient recurrence and not `InverseExpansionCoefficient`. It is the adapter's copying and replacing of fields after the positive-tail inversion has already succeeded. This is why the smallest repair belongs at object assembly.

The `Remainder` and `RemainderScaleExpression` fields describe magnitude scales. Their lack of a negative sign is correct; reflecting a source value does not make an absolute error bound negative. The repair must not indiscriminately multiply every inherited field by `sign`. Similarly, a missing frontier is absence of coefficient information, not a symbolic coefficient to be negated.

The concrete reproduction uses the default model order and cutoff one, where the displayed next coefficient is available. No broader claim is made here that every model-limited frontier is an exact next coefficient of the original special function. The sign error is established in a case where the independent original-function calculation supplies that coefficient.

4.4 Minimal patch

The supplied patch inserts the following field immediately after the `Erfc` adapter's expression and terms:

```
"FrontierTerm" -> If[MissingQ[innerData["FrontierTerm"]],
  innerData["FrontierTerm"], sign innerData["FrontierTerm"]],
```


The positive-source case is unchanged. A numerical or symbolic signed block is reflected at negative source infinity; `Missing[...]` is preserved. The guard is intentionally local and does not introduce a new metadata schema.

The native in-memory test first required the old assembly anchor to occur exactly once. After insertion, the same public probe returned the same finite expression and remainder power, with $-B(L)v^{-3/2}$ as its frontier. This is targeted validation of the proposed mechanism. It is not a claim that the canonical modular source was rebuilt into a standalone and then accepted by every upstream test.

4.5 Independent numerical evidence

The following table solves the original positive-tail equation at ninety working decimal digits and reflects the resulting root. It avoids subtracting a tiny tail from two by using v directly in $\log(\operatorname{erfc} z) + v + \frac{1}{2} \log \pi = 0$. Let $e = \operatorname{erfc}^{-1}(y) - g_-(y)$ and $F = B(L)v^{-3/2}$. The last two columns compare the errors after adding the stored or corrected frontier, respectively.

v	$e/(-F)$	$ e - F / e $	$ e + F / e $
16	0.979844	2.020571	0.020571
64	1.004952	1.995072	0.004928
256	1.002721	1.997287	0.002713
1024	1.000988	1.999013	0.000987

The second column approaches one, the incorrect correction approaches a factor-two worsening, and the corrected correction removes the leading error. These numerical values corroborate the exact coefficient derivation; they do not replace it with an interval certificate.

4.6 Integration acceptance for F01

The proposed `ReviewProbes.wlt` includes the negative-source witness, an unchanged positive-source control, preservation of the approximation and remainder power, and a reflected case with an affine target transformation. For the general target equation

$$y = \alpha + \beta \operatorname{erfc}(x), \quad \beta \neq 0,$$

source reflection corresponds to $y \mapsto 2\alpha + 2\beta - y$. The signed frontier must transform with the same source sign as the finite approximation, regardless of the sign of β .

The remaining acceptance work is specific: apply the canonical-source patch, regenerate the standalone using the repository's existing builder, and run those proposed cases in both layouts and both target engines. This report supplies the cases but does not report those unperformed runs as passing.

5 G01: a coupled Lerch limit is conservatively unsupported

5.1 The reproduced request

The following request was executed against the same native-loaded revision:

```
AsymptoticAnalysis'AsymptoticExpansion[
  LerchPhi[Exp[-1/x], 2, x], {x, Infinity, 4},
  "Backend" -> "Package"]
```

It returned `UnsupportedNativeCoefficient`, with an unexpanded `LerchPhi[Exp[-u], 2, 1/u]` coefficient in the positive local coordinate. The exact fresh symbol name is immaterial. The native importer does not admit that amplitude as a polynomial-logarithmic coefficient.

The dedicated Lerch path is designed for a fixed first parameter z and fixed s , with the large third argument as the varying quantity. Its admission restriction is appropriate for the theorem it uses. The reproduced failure therefore identifies a useful unsupported regime, not a reason to weaken that restriction.[\[4, 10\]](#)

5.2 Why the existing fixed-parameter series cannot just be reused

For fixed $|z| < 1$, expansion of $(a + n)^{-s}$ gives coefficients involving

$$M_j(z) = \sum_{n=0}^{\infty} n^j z^n.$$

When $z = e^{-\lambda/a}$ with fixed $\lambda > 0$, these moments grow as a^{j+1} times constants depending on λ . The nominal term $a^{-s-j} M_j(z)$ is consequently of order a^{1-s} for every fixed j . Terms that were an ordered asymptotic hierarchy at fixed z now contribute at the same leading scale. This is a change of regime, not an inadequate truncation cutoff.

Even the first term demonstrates the problem. At $\lambda = 1, s = 2$, the fixed- z leading expression after substitution is

$$\frac{a^{-2}}{1 - e^{-1/a}} \sim \frac{1}{a}.$$

The correct coefficient of a^{-1} is instead

$$J_2(1) = \int_0^{\infty} \frac{e^{-t}}{(1+t)^2} dt = e\Gamma(-1, 1) \approx 0.4036526377.$$

Thus extending the guard while retaining the old finite formula would give an incorrect leading coefficient. A new provider must sum this moving parameter's leading contribution before generating corrections.

The next section does exactly that. It does not change the existing fixed- z provider or reinterpret its parameter-only assumptions as uniformity claims.

6 E01: a uniform expansion with a signed next-term bound

6.1 Definitions and result

For $a > 0, \lambda > 0$, and $s \geq 0$, define

$$F(a; \lambda, s) = \Phi(e^{-\lambda/a}, s, a) = \sum_{n=0}^{\infty} \frac{e^{-\lambda n/a}}{(a + n)^s}.$$

The defining sum and the standard integral representation of Lerch's transcendent provide the starting point.[17] Let

$$J_s(\lambda) = \int_0^\infty e^{-\lambda t} (1+t)^{-s} dt = e^\lambda \lambda^{s-1} \Gamma(1-s, \lambda), \quad (6)$$

$$P_m(\lambda, s) = \sum_{j=0}^m \binom{m}{j} \lambda^{m-j} (s)_j, \quad P_0 = 1, \quad (7)$$

where $(s)_j$ is the rising factorial. At $s = 0$, (6) equals $1/\lambda$ and $P_m(\lambda, 0) = \lambda^m$. For $K \geq 0$, set

$$S_K(a) = a^{1-s} J_s(\lambda) + \frac{1}{2} a^{-s} + \sum_{k=1}^K \frac{B_{2k}}{(2k)!} P_{2k-1}(\lambda, s) a^{-s-2k+1}. \quad (8)$$

An empty sum is intended when $K = 0$.

Theorem 6.1 (Signed, globally valid remainder bound). *For every $a > 0$, $\lambda > 0$, $s \geq 0$, and integer $K \geq 0$,*

$$0 \leq (-1)^K (F(a; \lambda, s) - S_K(a)) \leq C_K(\lambda, s) a^{-s-2K-1}, \quad (9)$$

where

$$C_K(\lambda, s) = \frac{|B_{2K+2}|}{(2K+2)!} P_{2K+1}(\lambda, s). \quad (10)$$

In particular, the remainder has the sign of the first omitted Bernoulli term and is bounded in magnitude by that term.

This is an exact-expression theorem. Computing its endpoints in ordinary floating-point arithmetic does not by itself produce outward-rounded numerical intervals. The supplied prototype makes that distinction explicit.

6.2 A scalar kernel inequality

The proof can be reduced to an elementary positive-kernel remainder. Put

$$H(v) = \frac{1}{1 - e^{-v}}, \quad v > 0.$$

The classical partial fraction expansion of the hyperbolic cotangent gives

$$H(v) = \frac{1}{v} + \frac{1}{2} + 2v \sum_{n=1}^{\infty} \frac{1}{(2\pi n)^2 + v^2}. \quad (11)$$

This follows from $H(v) = \frac{1}{2} + \frac{1}{2} \coth(v/2)$ and the standard cotangent/cotangent-hyperbolic partial fractions.[18]

Lemma 6.2. *For $v > 0$ and integer $K \geq 0$,*

$$H(v) = \frac{1}{v} + \frac{1}{2} + \sum_{k=1}^K \frac{B_{2k}}{(2k)!} v^{2k-1} + r_K(v),$$

with

$$0 \leq (-1)^K r_K(v) \leq \frac{|B_{2K+2}|}{(2K+2)!} v^{2K+1}.$$

Proof. For $b > 0$, the finite geometric identity gives

$$\frac{1}{b^2 + v^2} = \sum_{j=0}^{K-1} \frac{(-1)^j v^{2j}}{b^{2j+2}} + \frac{(-1)^K v^{2K}}{b^{2K}(b^2 + v^2)}.$$

There is no requirement that $v/b < 1$: the identity is finite, and its remainder is exact. Substitution into (11) yields

$$r_K(v) = 2(-1)^K v^{2K+1} \sum_{n=1}^{\infty} \frac{1}{(2\pi n)^{2K}((2\pi n)^2 + v^2)}.$$

The summands are positive. Replacing the final denominator by $(2\pi n)^2$ bounds its magnitude by

$$\frac{2\zeta(2K+2)}{(2\pi)^{2K+2}} v^{2K+1} = \frac{|B_{2K+2}|}{(2K+2)!} v^{2K+1}.$$

The same even-zeta identity identifies the finite coefficients with the Bernoulli coefficients in the statement.[\[19\]](#) $\square$

The proof is valuable computationally because it avoids estimating a large number of tail terms independently. The sign and the first-neglected-term bound come from a positive denominator and survive subsequent integration.

6.3 Proof of the Lerch theorem

Proof of Theorem 6.1. First assume $s > 0$. The Gamma integral for $(a+n)^{-s}$ and positivity permit interchange of summation and integration. After the change of variable $r = at$, the Lerch integral becomes

$$F(a; \lambda, s) = \frac{a^{-s}}{\Gamma(s)} \int_0^{\infty} r^{s-1} e^{-r} H\left(\frac{r+\lambda}{a}\right) dr. \quad (12)$$

Thus the varying geometric parameter has become a shift by λ inside a scalar kernel averaged against a positive Gamma weight.

The $1/v$ term in the kernel contributes

$$\frac{a^{1-s}}{\Gamma(s)} \int_0^{\infty} \frac{r^{s-1} e^{-r}}{r+\lambda} dr = a^{1-s} J_s(\lambda).$$

For example, the equality with the integral in (6) follows by writing $(r+\lambda)^{-1} = \int_0^{\infty} e^{-(r+\lambda)t} dt$ and interchanging the positive integrals. The substitution $u = \lambda(1+t)$ then gives the incomplete Gamma representation in (6).

For each nonnegative integer m , the remaining polynomial moments satisfy

$$\frac{1}{\Gamma(s)} \int_0^{\infty} r^{s-1} e^{-r} (r+\lambda)^m dr = \sum_{j=0}^m \binom{m}{j} \lambda^{m-j} \frac{\Gamma(s+j)}{\Gamma(s)} = P_m(\lambda, s).$$

The finite kernel terms therefore give exactly (8). Its remainder has sign $(-1)^K$ because the averaging weight is positive. Averaging the scalar upper bound yields precisely (10) times $a^{-s-2K-1}$.

At $s = 0$, the defining sum is geometric: $F(a; \lambda, 0) = H(\lambda/a)$. The scalar lemma gives the statement directly, with $J_0(\lambda) = 1/\lambda$ and $P_m(\lambda, 0) = \lambda^m$. No singular Gamma density at $s = 0$ needs to be manipulated. $\square$

6.4 Uniformity and boundary checks

For fixed K , the bound is uniform when λ lies in a compact positive interval and s lies in a compact subset of $[0, \infty)$. Indeed, $P_{2K+1}(\lambda, s)$ is continuous and bounded on such a parameter set. The positive leading integral has a uniform lower bound, for example by restricting its defining integral to $0 \leq t \leq 1$. Consequently the relative remainder after the terms in (8) is uniformly $O(a^{-2K-2})$ on that compact set.

The theorem's absolute inequality holds for every positive a and λ , but that does not imply a useful small relative error when λ itself grows on the scale of a . This is why automatic dispatch must recognize a fixed transition parameter rather than merely detect an exponential expression in the first argument.

There are two useful exact consistency checks. At $s = 0$, the formula reduces to the Bernoulli expansion of the elementary geometric denominator. As $\lambda \downarrow 0$ with $s > 1$, $J_s(\lambda)$ tends to $1/(s-1)$ and $P_m(0, s) = (s)_m$, recovering the familiar large- a Hurwitz-zeta coefficient pattern. For $s \leq 1$, that leading integral diverges at $\lambda = 0$, so the prototype deliberately retains the strict restriction $\lambda > 0$. The boundary comparison is a consistency check, not an additional implemented provider claim.

6.5 A fully explicit example

For $\lambda = 1$ and $s = 2$, the first relevant moments are

$$P_1(1, 2) = 3, \quad P_3(1, 2) = 49, \quad P_5(1, 2) = 1631.$$

With $K = 2$,

$$S_2(a) = \frac{e\Gamma(-1, 1)}{a} + \frac{1}{2a^2} + \frac{1}{4a^3} - \frac{49}{720a^5}, \quad (13)$$

and

$$0 \leq \Phi(e^{-1/a}, 2, a) - S_2(a) \leq \frac{233}{4320a^7}. \quad (14)$$

The coefficient $233/4320$ is $1631/30240$ in reduced form. The native coefficient routine returned exactly these coefficients and this bound. At $a = 10$, its numerical ratio of error to upper bound was

$$0.985632727002140196667077137474\dots,$$

consistent with the independently computed table below.

a	K	$F - S_K$	Analytic bound $C_K a^{-s-2K-1}$	$(-1)^K(F - S_K)/\text{bound}$
5	0	1.9789×10^{-3}	2.0000×10^{-3}	0.989438
5	1	-2.1125×10^{-5}	2.1778×10^{-5}	0.970009
5	2	6.5313×10^{-7}	6.9037×10^{-7}	0.946064
10	0	2.4932×10^{-4}	2.5000×10^{-4}	0.997299
10	1	-6.7524×10^{-7}	6.8056×10^{-7}	0.992189
10	2	5.3160×10^{-9}	5.3935×10^{-9}	0.985633
50	0	1.9998×10^{-6}	2.0000×10^{-6}	0.999891
50	1	-2.1771×10^{-10}	2.1778×10^{-10}	0.999683
50	2	6.8996×10^{-14}	6.9037×10^{-14}	0.999413
100	0	2.4999×10^{-7}	2.5000×10^{-7}	0.999973
100	1	-6.8050×10^{-12}	6.8056×10^{-12}	0.999921
100	2	5.3927×10^{-16}	5.3935×10^{-16}	0.999853

These are ninety-digit numerical comparisons subsequently rounded for display. The proof supplies the mathematical inequality; the displayed floating-point values are not outward-rounded certificates.

7 Implementation and performance implications

7.1 The additive interface

The Wolfram companion exposes

```
AsymptoticAudit'UniformLerchModel[lambda, s, a, K]
```

in its own context. It returns a plain association containing the expression, terms in the coordinate $1/a$, the signed first omitted term, upper and lower remainder bounds, and validity conditions. It does not install a new definition on `LerchPhi`, mutate `System'` symbols, or pretend to be an already integrated `GeneralizedSeries` backend.

The prototype accepts positive rational λ , nonnegative rational s , an unset nonnumeric coordinate symbol, and $0 \leq K \leq 64$. These are deliberate implementation limits, not limits of Theorem 6.1. Exact-rational parameters make positivity and coefficient generation transparent across engines without invoking a broad assumption prover. The theorem itself admits arbitrary real parameters in its stated range.

The Python companion makes the same rational-parameter restriction and rejects floating-point inputs rather than silently claiming an exact parameter theorem for rounded data. It keeps exact coefficient generation separate from `mpmath` evaluation. Both companions explicitly deny an outward-rounded numerical certificate in their result metadata.

7.2 Coefficient construction

Equation (7) gives a finite nonnegative sum for every moment in the admitted parameter range. This is the reference algorithm used by the prototype. It is straightforward to test against the recurrence

$$P_{m+1} = (\lambda + s + m)P_m - \lambda m P_{m-1}, \quad P_0 = 1, \quad P_1 = \lambda + s. \quad (15)$$

To derive it, differentiate the moment-generating function $e^{\lambda t}(1-t)^{-s}$, multiply by $1-t$, and compare coefficients. The exact independent tests compare the finite sum and recurrence for twelve rational parameter pairs through degree ten.

For exact arithmetic the recurrence can reduce repeated construction work. For floating-point arithmetic its subtraction can be undesirable, especially near $s = 0$, where substantial cancellation can leave a simple λ^{m+1} . The positive finite sum is therefore retained as the numerical reference rather than replaced by a recurrence solely because it has fewer arithmetic operations. A future integrated implementation can choose between them using measured coefficient size and working-precision behavior.

7.3 Work as the large parameter grows

Generating all required moments by finite sums takes quadratically many scalar arithmetic steps in K , apart from integer/rational bit complexity. Evaluation of the resulting correction polynomial uses $O(K)$ terms and one leading integral or incomplete-Gamma value. Increasing a does not increase the number of coefficients at a fixed K .

This is the concrete performance opportunity: the coupled regime need not be handled by summing a geometric tail whose effective length grows with a , or by repeatedly computing fixed- z moments that all contribute at the same leading order. It is not an empirical claim that the current package is slower by a measured factor. No benchmark of the upstream package against the new prototype was run.

A useful implementation should also separate coefficient caching from numerical lead-term evaluation. The cache key for the polynomial data is (λ, s, K) with exact parameter semantics; the large coordinate a is not part of that data. The prototype bounds its supported order and does not claim to be a general resource sandbox for enormous rational inputs.

7.4 Numerical stability of the leading term

The symbolic representation $e^\lambda \lambda^{s-1} \Gamma(1-s, \lambda)$ is compact and exact. Its factorization can be numerically awkward for large λ because separately large and small factors compensate. The equivalent positive integral in (6) is a useful independent reference. For small λ , behavior near $s = 1$ must also be handled without claiming uniformity at a singular endpoint that the theorem excludes.

The present Python evaluator uses arbitrary-precision incomplete Gamma arithmetic and does not certify the rounding error of that evaluation. A production numerical enclosure would additionally need a rigorously bounded value of $J_s(\lambda)$ and outward-rounded evaluation of the finite terms. That is a distinct deliverable from the analytic tail bound already proved here. The result's `interval_certificate` and `OutwardRounded` fields remain false for precisely this reason.

7.5 A bounded integration path

The first integration step should be an explicit transition provider whose input states that λ and s are fixed. It should preserve the existing fixed- z admission guard. Once a dedicated test matrix passes, the outer dispatcher may recognize an exact expression of the form `LerchPhi [Exp [-lambda/a], s, a]` with proved fixed parameters. An arbitrary expression in the first argument is not sufficient evidence of this regime.

Mapping the companion association into `GeneralizedSeries` requires a recorded positive scale $1/a$, the leading coefficient $J_s(\lambda)$, the source function with its fixed-parameter contract, and the signed analytic bound. It must not automatically acquire an all-orders derivative permission merely because its magnitude tail is known. No such permission is installed by the companion code.

The explicit K interface is intentionally separate from a request for a number of nonzero exponent blocks: the Bernoulli pattern omits many powers. An integrated term-goal interface must count actual retained blocks rather than silently identify K with a generic power cutoff. This recommendation is confined to the new provider's API design.

8 Wolfram 15 and Mathics3 assessment

The observed Wolfram result establishes that the pinned generated standalone loads and executes the particular requests recorded in the evidence file. It does not establish that every advertised feature works under version 15. The native built-in asymptotic facility provides a useful reference interface, but its availability is not evidence that a package's explicit analytic contract, branch record, or copied metadata is correct.[20]

The pinned Mathics document explicitly describes complete compatibility as a project goal that has not yet been achieved. It distinguishes selected loading and acceptance milestones from comprehensive public-operation coverage. This review adopts that evidence distinction rather than overriding it with a new compatibility label.[11]

For F01, the defect is in common adapter object assembly. Nothing in the inspected `MathicsSpecialFunctions.wl` fallback repairs that field; that module adds a large-positive `LogGamma` path instead. However, the original `Erfc` reproduction must still be run in Mathics before asserting its observed end-to-end manifestation there. Source sharing identifies where to test; it does not make an unperformed execution happen.[12]

The E01 companion avoids relying on a native joint Lerch asymptotic expansion. Its coefficient path uses finite integer loops, exact rational arithmetic, Bernoulli numbers, binomial coefficients and rising factorials. Its leading incomplete-Gamma expression may remain symbolic in an engine without the needed numerical support; that is acceptable for an exact symbolic model but not for a claimed numerical enclosure. The distributed WL file and proposed WLT cases remain Mathics-unverified.

The native and Mathics acceptance work should therefore be reported in three separate categories: coefficient/formula construction, successful evaluation of required special-function constants, and validated numerical or interval comparison with the original function. Combining these into one “works” flag would obscure the new provider’s actual capabilities. This is an acceptance plan for E01, not a reclassification of the existing repository-wide ledger.

9 Priorities and concrete deliverables

The immediate change is F01’s local field replacement. It is small enough to review directly and targets a reproduced contract violation. The acceptance criterion is reflection covariance of the signed frontier together with unchanged finite approximation and magnitude remainder. The prepared patch and the proposed integration cases are in the archive.

The next development item is G01/E01: add the transition expansion as an explicit provider, initially within the theorem’s positive-parameter range. The delivered theorem, exact coefficients, signed bound and independent tests supply more than a request to “support more special functions.” They define a particular admissible input family and a concrete mathematical contract.

Only after that provider is accepted should automatic recognition of the moving-parameter pattern be enabled. Recognition must establish fixed λ and s ; its success must not relax the original provider’s fixed-parameter assumptions. Numerical interval certification, broader parameter ranges and optimized moment evaluation are separate follow-on milestones with separate acceptance requirements.

No additional high-severity defect is asserted on the basis of the source areas where this review did not establish one. The practical conclusion is therefore deliberately specific: repair the reflected signed frontier, retain conservative failure for the unsupported transition until its provider exists, and evaluate the supplied transition implementation under the two target engines without promoting independent tests into upstream acceptance evidence.

A Artifact inventory and reproduction

Artifact	Purpose
article/article.tex, article/article.pdf	Editable report and compiled article.
patches/erfc-frontier.patch	Minimal canonical-source F01 insertion.
code/prepare_erfc_patch.py	Emit a diff after checking one exact assembly anchor; does not modify a checkout or verify its revision.
code/uniform_lerch.py	Exact-rational model generator and clearly non-interval numerical evaluator.
code/UniformLerch.wl	Additive Wolfram Language prototype in a separate context; no upstream definitions replaced.
code/ReviewProbes.wlt	Proposed integration tests. Complete file not executed in this review.
code/test_review.py	Twelve independent test methods, including 96 numerical signed-bound subcases.
code/make_tables.py	Regenerate the independent numerical tables.
evidence/native-observations.md	Native inputs, transcribed outputs, and explicit rejected/incomplete execution scope.
evidence/novelty-ledger.csv	Claim-specific non-overlap rationale.
evidence/environment.json	Observed local environment and major unperformed validation steps.

From the archive root, the independent checks and an example evaluation are:

```
python -m pip install -r code/requirements.txt
python code/test_review.py
python code/make_tables.py
python code/uniform_lerch.py --lambda 1 --s 2 --order 2 --a 10 --dps 80
```

The actual execution transcript is retained; rerunning the commands does not turn these into native or Mathics upstream tests.

To prepare a patch against a local checkout:

```
python code/prepare_erfc_patch.py /path/to/Asymptotic > erfc-frontier.patch
```

The generator refuses a missing, duplicated or already-patched anchor. It is an exact-fragment guard, not a source-integrity verifier. After reviewing and applying the canonical change, use the repository's existing standalone builder and test runners rather than editing only the generated root file.

The PDF can be rebuilt from article/ with three ordinary `pdflatex-interaction=nonstopmode-halt-on-error` article.tex runs. The numerical-table inputs are included. No checksum files or font files are part of the distribution.

B The independent test matrix

The twelve method names in evidence/independent-tests.txt correspond to separate categories, not a hidden claim of comprehensive package testing. The exact checks verify Bernoulli constants against an independent CAS, Gamma-moment recurrence identities, the

finite geometric-remainder identity, the explicit example's coefficients, the $s = 0$ limit, and the Erfc coefficient equations. Guard tests cover the patch generator and companion input contract.

The 96 numerical signed-bound subcases use

$$\lambda \in \{1/3, 1, 3\}, \quad s \in \{0, 1, 2, 3/2\}, \quad a \in \{10, 50\}, \quad K \in \{0, 1, 2, 3\}.$$

Each checks the numerical ratio in (9), with a very small tolerance for numerical evaluation. The proof, not those samples, establishes the inequality outside the grid. Four independent original-Erfc root comparisons check the wrong-versus-correct correction behavior. A final numerical control shows that the fixed- z first term has the wrong leading coefficient after substitution onto the transition curve.

The full proposed WLT integration file, negative-scale affine Erfc control, Mathics execution, modular-layout execution, and rebuilt standalone acceptance remain work to perform. These omissions are stated in the article and in the archive README so that the delivered files cannot reasonably be mistaken for a completed cross-engine acceptance release.

References

- [1] Vladimir Reshetnikov, *Asymptotic code-review index*, reviewed revision efa1aee, including the six linked wave indexes. <https://github.com/VladimirReshetnikov/Asymptotic/tree/efa1aee4845a9c35e140963a0333d0c9ec33b05/external-reports/code-review>.
- [2] *Code review status*, same revision. https://github.com/VladimirReshetnikov/Asymptotic/blob/efa1aee4845a9c35e140963a0333d0c9ec33b05/docs/development/CODE_REVIEW_STATUS.md.
- [3] *SpecialFunctionAdapters.wl*, especially Erfc construction, lines 20–87 and assembly lines 47–68, same revision. <https://github.com/VladimirReshetnikov/Asymptotic/blob/efa1aee4845a9c35e140963a0333d0c9ec33b05/src/Kernel/SpecialFunctionAdapters.wl>.
- [4] *DirichletSpecialFunctions.wl*, same revision. <https://github.com/VladimirReshetnikov/Asymptotic/blob/efa1aee4845a9c35e140963a0333d0c9ec33b05/src/Kernel/DirichletSpecialFunctions.wl>.
- [5] *Canonical AsymptoticAnalysis.wl*, same revision; relative-cutoff validation in the inverse constructor. <https://github.com/VladimirReshetnikov/Asymptotic/blob/efa1aee4845a9c35e140963a0333d0c9ec33b05/src/Kernel/AsymptoticAnalysis.wl>.
- [6] *SeriesOperations.wl*, same revision. <https://github.com/VladimirReshetnikov/Asymptotic/blob/efa1aee4845a9c35e140963a0333d0c9ec33b05/src/Kernel/SeriesOperations.wl>.
- [7] *BarnesForward.wl*, same revision. <https://github.com/VladimirReshetnikov/Asymptotic/blob/efa1aee4845a9c35e140963a0333d0c9ec33b05/src/Kernel/BarnesForward.wl>.
- [8] *SpecialFunctionIdentities.wl*, same revision. <https://github.com/VladimirReshetnikov/Asymptotic/blob/efa1aee4845a9c35e140963a0333d0c9ec33b05/src/Kernel/SpecialFunctionIdentities.wl>.
- [9] *ParameterizedSpecialFunctions.wl*, same revision. <https://github.com/VladimirReshetnikov/Asymptotic/blob/efa1aee4845a9c35e140963a0333d0c9ec33b05/src/Kernel/ParameterizedSpecialFunctions.wl>.
- [10] *NativeSpecialFunctions.wl*, same revision. <https://github.com/VladimirReshetnikov/Asymptotic/blob/efa1aee4845a9c35e140963a0333d0c9ec33b05/src/Kernel/NativeSpecialFunctions.wl>.
- [11] *Running AsymptoticAnalysis in Mathics3*, same revision. <https://github.com/VladimirReshetnikov/Asymptotic/blob/efa1aee4845a9c35e140963a0333d0c9ec33b05/docs/Mathics/COMPATIBILITY.md>.

- [12] *MathicsSpecialFunctions.wl*, same revision. <https://github.com/VladimirReshetnikov/Asymptotic/blob/efa1aeec4845a9c35e140963a0333d0c9ec33b05/src/Kernel/MathicsSpecialFunctions.wl>.
- [13] *Code review 43: focused source-review deltas*, README and claim-specific scope, retained in wave 5 at the reviewed revision. <https://github.com/VladimirReshetnikov/Asymptotic/blob/efa1aeec4845a9c35e140963a0333d0c9ec33b05/external-reports/code-review/wave-5/code-review-43/README.md>.
- [14] *Code review 49: mechanism-level novelty crosswalk*, retained in wave 6 at the reviewed revision. https://github.com/VladimirReshetnikov/Asymptotic/blob/efa1aeec4845a9c35e140963a0333d0c9ec33b05/external-reports/code-review/wave-6/code-review-49/evidence/novelty_crosswalk.md.
- [15] NIST Digital Library of Mathematical Functions, §7.4, especially equation 7.4.2, Erfc reflection. <https://dlmf.nist.gov/7.4.E2>.
- [16] NIST DLMF, §7.12(i), positive-real Erfc asymptotics. <https://dlmf.nist.gov/7.12.i>.
- [17] NIST DLMF, §25.14, especially equations 25.14.1 and 25.14.5, defining series and integral representation of Lerch's transcendent. <https://dlmf.nist.gov/25.14>.
- [18] NIST DLMF, §4.36, infinite products and partial fractions for hyperbolic functions. <https://dlmf.nist.gov/4.36>.
- [19] NIST DLMF, equation 25.6.2, even zeta values and Bernoulli numbers. <https://dlmf.nist.gov/25.6.E2>.
- [20] Wolfram Research, *Asymptotic*, Wolfram Language documentation. <https://reference.wolfram.com/language/ref/Asymptotic.html>.